

Module 3

Hadoop

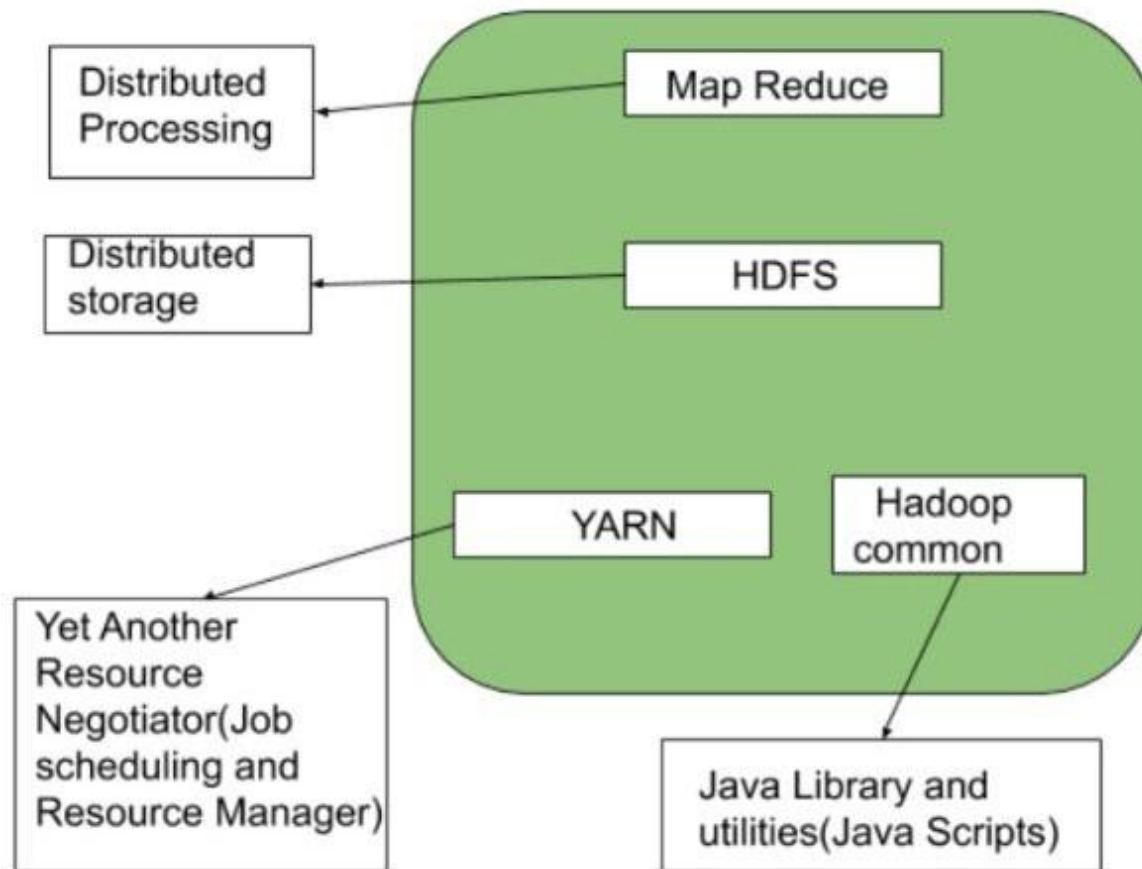
- Hadoop is an Apache open source framework written in java.
- Hadoop allows distributed processing of large datasets across clusters of computers using simple programming models.
- The Hadoop framework application works in an environment that provides distributed storage and computation across clusters of computers.
- Hadoop is designed to scale up from single server to thousands of machines, each offering local computation and storage.

Evolution of Hadoop

- Hadoop was designed by **Doug Cutting and Michael Cafarella** in 2005 inspired by the Google.
- Hadoop stores the huge amount of data through a system called Hadoop Distributed File System (HDFS) and processes this data with the technology of Map Reduce.
- The designs of HDFS and Map Reduce are inspired by the Google File System (GFS) and Map Reduce.
- In the year 2003 Google released papers on GFS and in 2004, released another paper on Map Reduce.
- The two enthusiasts Doug Cutting and Michael Cafarella studied those papers and designed what is called, Hadoop in the year 2005

Hadoop Architecture

- The Hadoop Architecture Mainly consists of 4 components.
 1. MapReduce
 2. HDFS(Hadoop distributed File System)
 3. YARN(Yet Another Resource Framework)
 4. Common Utilities or Hadoop Common



HDFS

- HDFS(Hadoop Distributed File System) is utilized for storage in a Hadoop cluster.
- It is used to scale a single Apache Hadoop cluster to hundreds (and even thousands) of nodes.
- HDFS is designed in such a way that ,more in storing the data in a large chunk of blocks rather than storing small data blocks.
- HDFS in Hadoop provides Fault-tolerance and High availability to the storage layer and the other devices present in that Hadoop cluster.

Goals of HDFS

Fast recovery from hardware failures

- In a single HDFS setup, there could be thousands of servers, and it's certain that at least one server might face issues or fail.
- HDFS has been built to detect faults and automatically recover quickly.

Access to streaming data

- Streaming data is data that is generated continuously by thousands of data sources, which typically send in the data records simultaneously, and in small sizes (order of Kilobytes).
- Streaming data includes a wide variety of data such as log files generated by customers using your mobile or web applications, ecommerce purchases, in-game player activity, information from social networks, financial trading floors etc

Accommodation of large data sets

- HDFS accommodates applications that have data sets typically gigabytes to terabytes in size.
- HDFS provides high aggregate data bandwidth and can scale to hundreds of nodes in a single cluster.

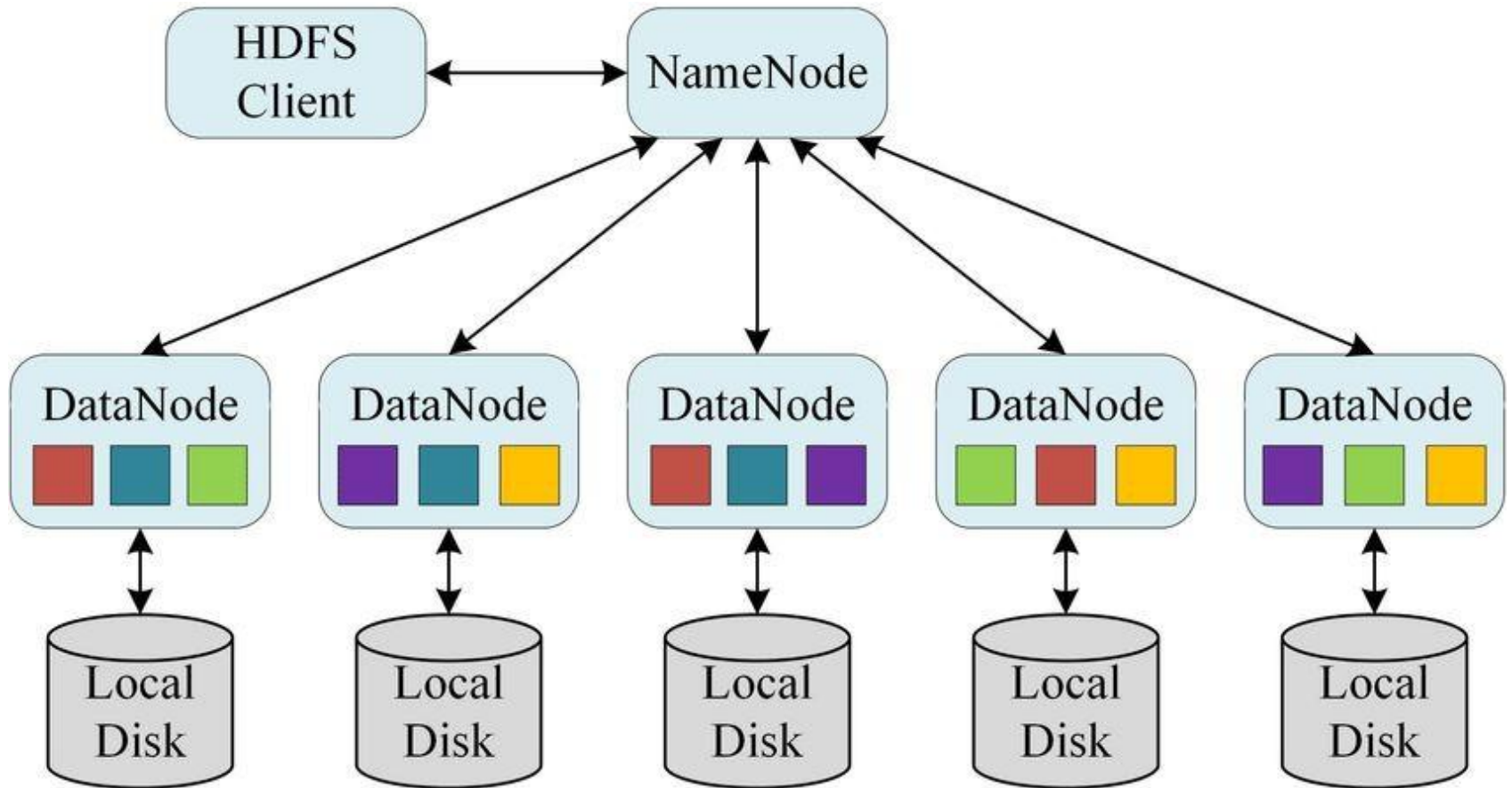
Portability

- HDFS is designed to be portable across multiple hardware platforms and to be compatible with a variety of underlying operating systems.

Context

- Consider a file that includes the phone numbers for everyone in the United States; the numbers for people with a last name starting with A might be stored on server 1, B on server 2, and so on.
- With Hadoop, pieces of this phonebook would be stored across the cluster, and to reconstruct the entire phonebook, our program would need the blocks from every server in the cluster.
- To ensure availability if and when a server fails, HDFS replicates these smaller pieces onto two additional servers by default.
- The redundancy can be increased or decreased on a per-file basis or for a whole environment.

HDFS

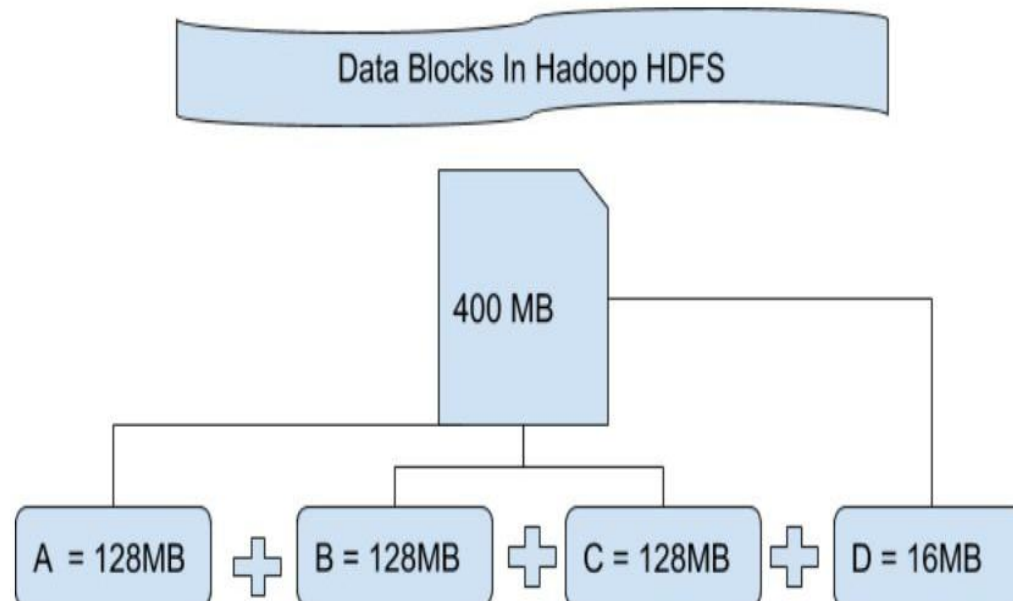


HDFS Concepts

Blocks

- A Block is the minimum amount of data that it can read or write.
- HDFS blocks are 128 MB by default and this is configurable.
- Files In HDFS are broken into block-sized chunks, which are stored as independent units.
- If the file in HDFS is smaller than block size, then it does not occupy full block's size,
- i.e. 5 MB of file stored in HDFS of block size 128 MB takes 5MB of space only.

- Suppose you have uploaded a file of 400MB to your HDFS then what happens is this file got divided into blocks of $128\text{MB} + 128\text{MB} + 128\text{MB} + 16\text{MB} = 400\text{MB}$ size.
- Means 4 blocks are created each of 128MB except the last one.



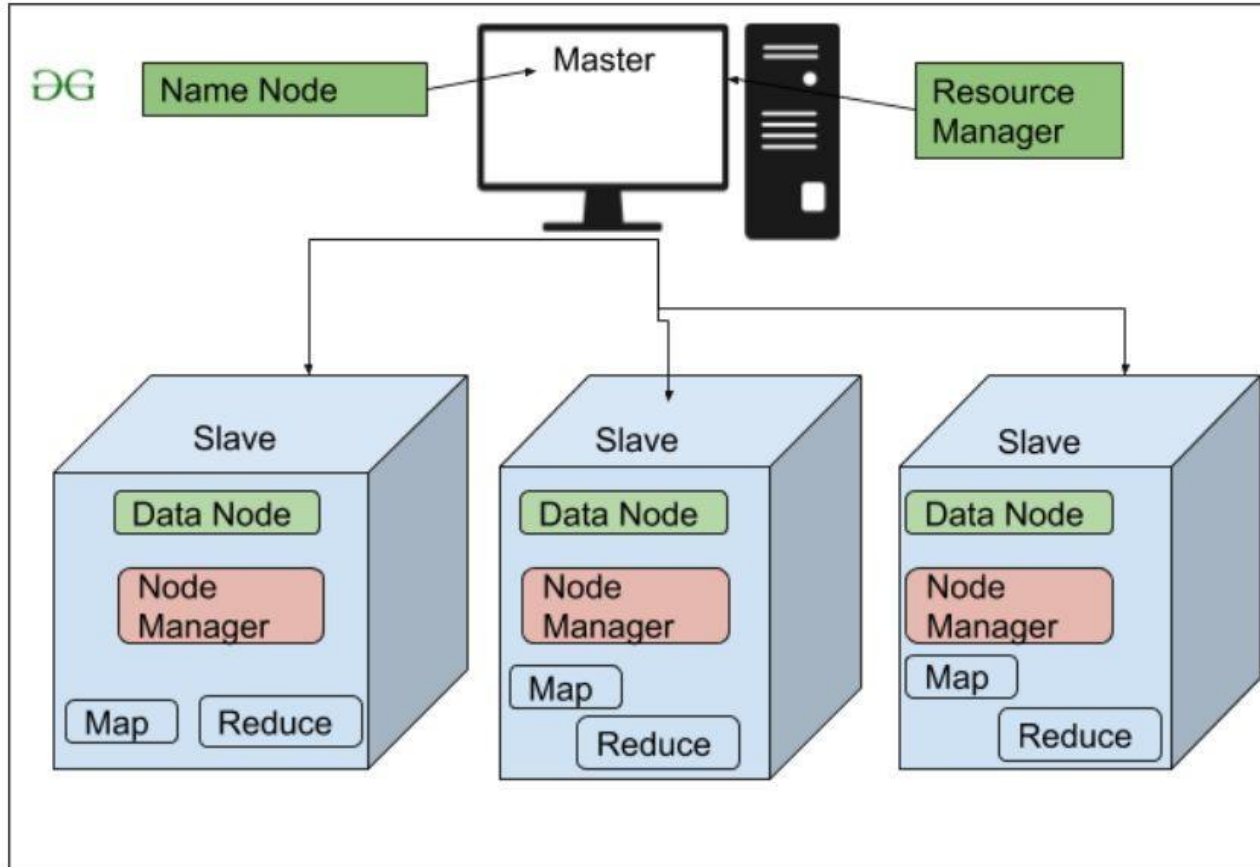
Name Node

- HDFS works in master-worker pattern where the name node acts as master.
- Name Node is controller and manager of HDFS as it knows the status and the metadata of all the files in HDFS
 - The metadata information being file permission, names location of each block, transaction logs
 - The metadata are small, so it is stored in the memory of name node, allowing faster access to data.
- HDFS cluster is accessed by multiple clients concurrently, so all this information is handled by single machine.
- The file system operations like opening, closing, renaming etc. are executed by it.

Data Node

- DataNodes works as a slave, data nodes are for storing the data in a Hadoop cluster.
- The number of data nodes can be from 1 to 500 or even more than that.
- They store and retrieve blocks when they are instructed to do by client or name node.
- They report back to namenode periodically, with list of blocks that they are storing.
- The data node being a commodity hardware also does the work of block creation, deletion and replication as stated by the name node.

High Level Architecture Of Hadoop



Replication In HDFS

- Replication ,making a copy of, ensures the availability of the data.
- The number of times we make a copy of that particular thing can be expressed as it's Replication Factor.
- By default, the Replication Factor for Hadoop is set to 3 which can be configured means we can change it manually.
- HDFS stores the data in the form of various blocks at the same time Hadoop is also configured to make a copy of those file blocks.

Why replication in hadoop ?

- We are using commodity hardware (inexpensive system hardware) which can be crashed at any time, that is why we need such a feature in HDFS which can make copies of that file blocks for backup purposes, this is known as fault tolerance.
- We can configure the replication factor in your ***hdfs-site.xml*** file.

Basic Filesystem Operations

- There are many interfaces to HDFS available, but the command line (**CLI**) is one of the simplest and, to many developers, the most familiar interface.
- You can perform most advanced and basic Hadoop HDFS filesystem operations using CLI.
- When Hadoop HDFS filesystem is set, you can do all of the basic HDFS filesystem operations, such as reading files, creating directories, moving files, deleting data, and listing directories.
- You can also perform the advance Hadoop HDFS filesystem operations such as updates, administrator from command line.

Hadoop commands for basic file system operations.

1. **ls:** This command is used to list all the files. Use `lsr` for recursive approach. It is useful when we want a hierarchy of a folder.

Syntax: `bin/hdfs dfs -ls`

Example: `bin/hdfs dfs -ls /`

It will print all the directories present in HDFS.

2. **mkdir:** To create a directory. In Hadoop dfs there is no home directory by default. So let's first create it.

Syntax: `bin/hdfs dfs -mkdir`
creating home directory:

hdfs/bin -mkdir /user

hdfs/bin -mkdir /user/username

Example: bin/hdfs dfs -mkdir /geeks

3. **touchz:** It creates an empty file.

Syntax: bin/hdfs dfs -touchz

Example: bin/hdfs dfs -touchz /geeks/myfile.txt

4. **copyFromLocal** (or) **put:** To copy files/folders from local file system to hdfs store. This is the most important command. Local filesystem means the files present on the OS.

Syntax: bin/hdfs dfs -copyFromLocal

Example: Let's suppose we have a file Al.txt on Desktop which we want to copy to folder geeks present on hdfs.

```
bin/hdfs dfs -copyFromLocal ../Desktop/Al.txt /geeks
```

(OR)

```
bin/hdfs dfs -put ../Desktop/Al.txt /geeks
```

5. **cat:** To print file contents.

Syntax: bin/hdfs dfs -cat

Example: // print the content of Al.txt present // inside geeks folder.

```
bin/hdfs dfs -cat /geeks/AI.txt
```

6. **copyToLocal** (or) **get**: To copy files/folders from hdfs store to local file system.

Syntax: bin/hdfs dfs -copyToLocal < < srcfile(on hdfs) > <local file dest>

Example: bin/hdfs dfs -copyToLocal /geeks ../Desktop/hero

(OR)

```
bin/hdfs dfs -get /geeks/myfile.txt ../Desktop/hero
```

7. **moveFromLocal:** This command will move file from local to hdfs.

Syntax: bin/hdfs dfs -moveFromLocal

Example: bin/hdfs dfs -moveFromLocal ../Desktop/cutAndPaste.txt
/geeks

8. **cp:** This command is used to copy files within hdfs. Lets copy folder geeks to geeks_copied.

Syntax: bin/hdfs dfs -cp

Example: bin/hdfs dfs -cp /geeks /geeks_copied

9. **mv:** This command is used to move files within hdfs. Lets cut-paste a file myfile.txt from geeks folder to geeks_copied.

Syntax: bin/hdfs dfs -mv

Example: bin/hdfs dfs -mv /geeks/myfile.txt /geeks_copied

10. **rmr:** This command deletes a file from HDFS recursively. It is very useful command when you want to delete a non-empty directory.

Syntax: bin/hdfs dfs -rmr

Example: bin/hdfs dfs -rmr /geeks_copied -> It will delete all the content inside the directory then the directory itself.

11. **du:** It will give the size of each file in directory.

Syntax: bin/hdfs dfs -du

Example: bin/hdfs dfs -du /geeks

12. **du**: This command will give the total size of directory/file.

Syntax: bin/hdfs dfs -du

Example: bin/hdfs dfs -du /geeks

13. **stat**: It will give the last modified time of directory or path. In short it will give stats of the directory or file.

Syntax: bin/hdfs dfs -stat

Example: bin/hdfs dfs -stat /geeks

14. **setrep:** This command is used to change the replication factor of a file/directory in HDFS. By default it is 3 for anything which is stored in HDFS (as set in hdfs core-site.xml).

Example 1: To change the replication factor to 6 for geeks.txt stored in HDFS.

```
bin/hdfs dfs -setrep -R -w 6 geeks.txt
```

Example 2: To change the replication factor to 4 for a directory geeksInput stored in HDFS.

```
bin/hdfs dfs -setrep -R 4 /geeks
```

Note: The -w means wait till the replication is completed. And -R means recursively, we use it for directories as they may also contain many files and folders inside them.

Hadoop File Systems

- Hadoop is capable of running various file systems and HDFS is just one single implementation that out of all those file systems.
- The Hadoop has a variety of file systems that can be implemented.
- The Java abstract class ***org.apache.hadoop.fs.FileSystem*** represents a file system in Hadoop.

Filesystem	URI scheme	Java implementation (all under <code>org.apache.hadoop</code>)	Description
Local	<i>file</i>	<code>fs.LocalFileSystem</code>	A filesystem for a locally connected disk with client-side checksums. Use <code>RawLocalFileSystem</code> for a local filesystem with no checksums. See “LocalFileSystem” on page 82 .
HDFS	<i>hdfs</i>	<code>hdfs.DistributedFileSystem</code>	Hadoop’s distributed filesystem. HDFS is designed to work efficiently in conjunction with MapReduce.
HFTP	<i>hftp</i>	<code>hdfs.HftpFileSystem</code>	A filesystem providing read-only access to HDFS over HTTP. (Despite its name, HFTP has no connection with FTP.) Often used with <i>distcp</i> (see “Parallel Copying with distcp” on page 75) to copy data between HDFS clusters running different versions.
HSFTP	<i>hsftp</i>	<code>hdfs.HsftpFileSystem</code>	A filesystem providing read-only access to HDFS over HTTPS. (Again, this has no connection with FTP.)
WebHDFS	<i>webhdfs</i>	<code>hdfs.web.WebHdfsFile</code>	A filesystem providing secure read-write access to HDFS

HAR	<i>har</i>	<code>fs.HarFileSystem</code>	A filesystem layered on another filesystem for archiving files. Hadoop Archives are typically used for archiving files in HDFS to reduce the namenode's memory usage. See "Hadoop Archives" on page 77 .
KFS (Cloud-Store)	<i>kfs</i>	<code>fs.kfs.KosmosFileSystem</code>	CloudStore (formerly Kosmos filesystem) is a distributed filesystem like HDFS or Google's GFS, written in C++. Find more information about it at http://code.google.com/p/kosmosfs/ .
FTP	<i>ftp</i>	<code>fs.ftp.FTPFileSystem</code>	A filesystem backed by an FTP server.
S3 (native)	<i>s3n</i>	<code>fs.s3native.NativeS3FileSystem</code>	A filesystem backed by Amazon S3. See http://wiki.apache.org/hadoop/AmazonS3 .

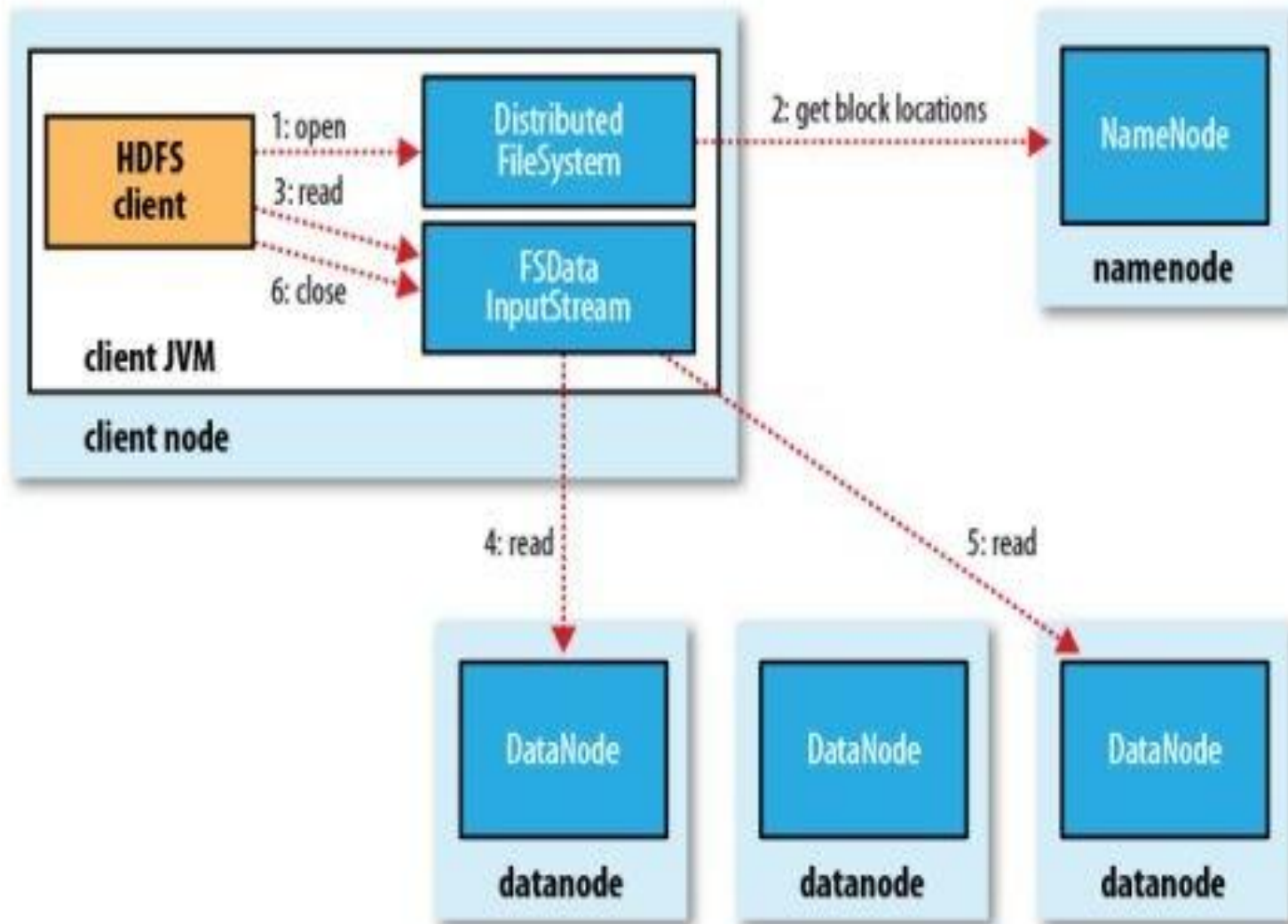


Figure 3-2. A client reading data from HDFS

Diagram shows the main sequence of events between client and HDFS when reading a file

FSDataInputStream

- This class is a specialization of **java.io.DataInputStream** class with support of random access, can read from any part of the stream.

DFSInputStream

- Input stream backed by a file in the distributed file system
- Read calls on a DFSInputStream are directed to a storage server hosting the given file.

Steps

- ❑ The client open the file it wishes to read using **open()** on the **FileSystem** object (step 1).
- ❑ **DistributedFileSystem** calls the **namenode** using RPC to determine the locations of the blocks for the first few blocks in the file (step 2).
- ❑ For each block the **namenode** returns the address of **datanodes** that have a copy of the block.

- ❑ The **datanodes** are sorted according to their proximity with client.
- ❑ If client itself is a **datanode** the client will reads from the local **datanode**.
- ❑ The **DistributedFileSystem** returns an **FSDataInputStream** to the client for it to read data from.
- ❑ **FSDataInputStream** in turn wraps a **DFSInputStream** which manages the **datanode** and **namenode**.

- ❑ The client then calls the **read()** on the stream **DFSInputStream** (step 3), which has stored data node addresses for the first few blocks in the file, then connects to the first **datanode**(closest) for the first block in the file.
- ❑ Data is streamed from the **datanode** back to the client, which calls **read()** repeatedly on the stream. (step 4)
- ❑ When the end of the block is reached, **DFSInputStream** will close the connection to the data node, then find the best **datanode** for the next block (step 5).

- ❑ Blocks are read in order, with **DFSInputStream** opening new connections to **datanodes** as the client reads through the stream
- ❑ It will also call the **namenode** to retrieve the **datanode's** locations for the next batch of blocks as needed.
- ❑ When the client has finished reading, it call **close()** on the **FSDatInputStream** (step 6).

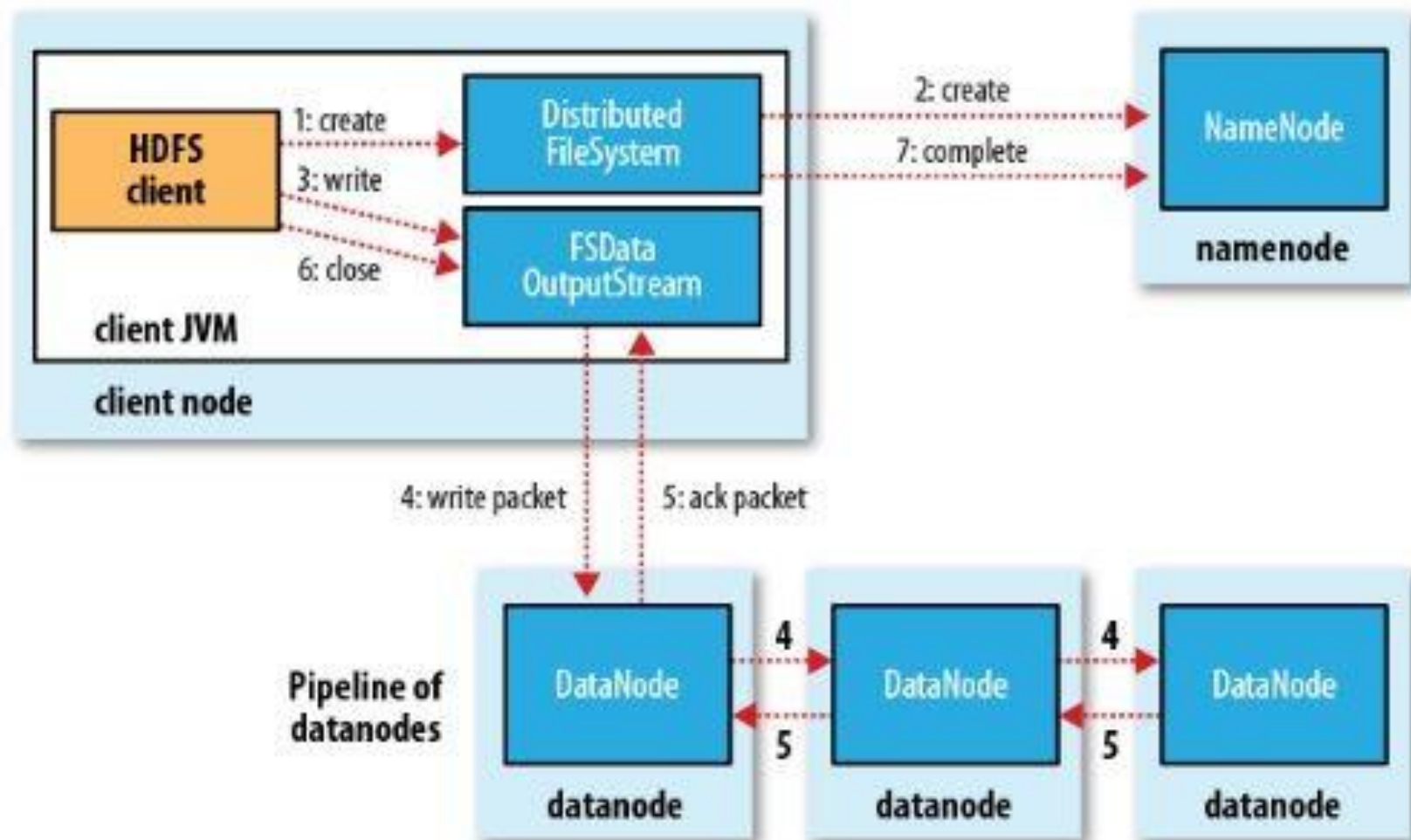


Figure 3-4. A client writing data to HDFS

- ❑ Here the set of operations included are :
 - Creating new file
 - Writing data
 - Closing the file
- ❑ The client create file by calling **create()** function on **DistributedFileSystem** (step 1).
- ❑ **DistributedFileSystem** makes an RPC call to the **namenode** to create a new file in the filesystem's name space (step 2).
- ❑ The **namenode** performs various checks to make sure the file does not already exist and client has the right permission to create file

- ❑ If check pass ,**namenode** makes a record of new file otherwise makes an **IOException** error.
- ❑ The **DistributedFileSystem** returns an **FSDataOutputStream** for the client to start write data to.
- ❑ **FSDataOutputStream** wraps a **DFSOutputStream**, which handles communication with the **namenode** and **datanode**.
- ❑ The client starts writing data, **DFSOutputStream** splits it into packets and writes to an internal queue called *data queue* (step 3).

- ❑ The data queue is consumed by **DataStreamer**, which is responsible for asking **namenode** to allocate new blocks by picking a list of **datanodes** to replicates.
- ❑ The list of datanodes forms a pipeline.
- ❑ The **DataStreamer** streams the packets to the first datanode in the pipeline, which stores and forwards to second and second to third (step 4)
- ❑ **DFSOutputStream** also keeps an internal queue of packets that are waiting to be acknowledged by datanodes, *ack queue*.
- ❑ A packet is removed from queue once its acknowledged by all datanodes in pipeline (step 5).

- ❑ When client finished writing it calls **close()** on the stream (step 6).
- ❑ This action flushes all packets to the datanodes pipeline and waits for acknowledgments before contacting the name node to signal that file is complete (step7).

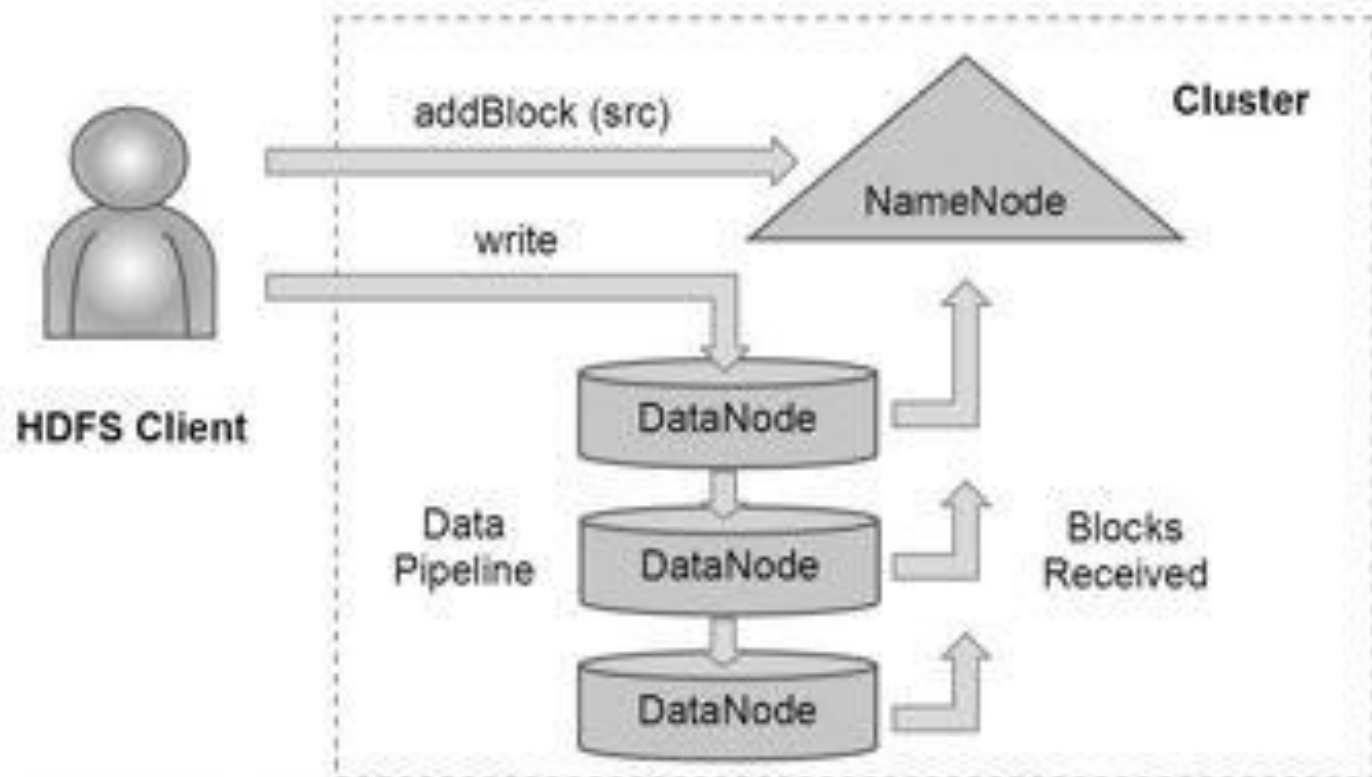


Figure 8.1: HDFS Client Creates a New File

MapReduce framework

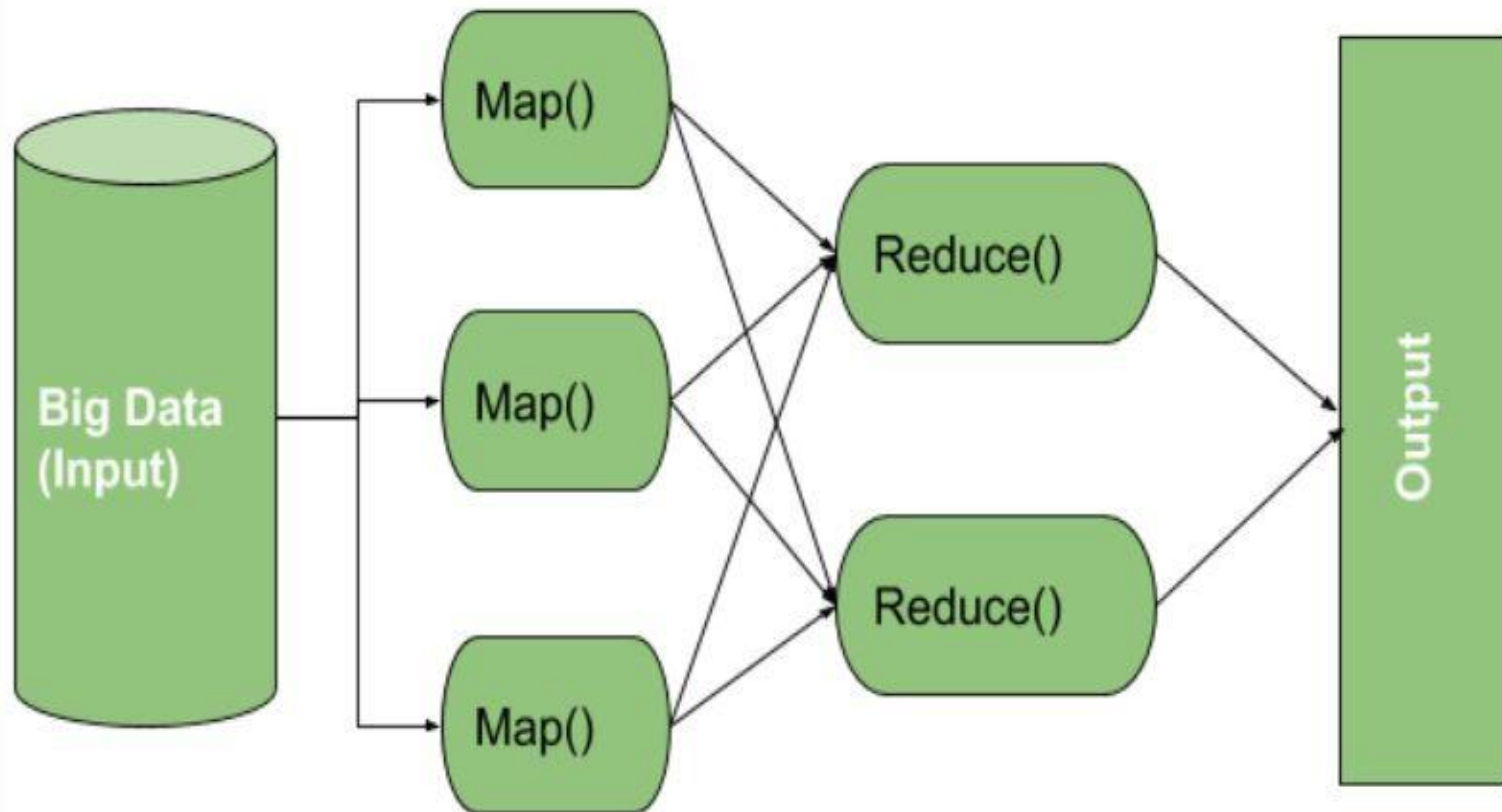
- The MapReduce framework is a programming model and processing paradigm designed for large-scale data processing on distributed computing clusters.
- It was popularized by Google and later implemented in open-source frameworks like Apache Hadoop.
- The MapReduce model simplifies the processing of vast amounts of data by breaking it down into smaller chunks and distributing the processing across multiple nodes in a cluster.
- MapReduce has mainly 2 tasks which are divided phase-wise:
 - Map
 - Reduce

- **Map stage**

- With this stage, the input file is passed to the mapper function line by line and the mapper processes the data and creates several small chunks of data.

- **Reduce stage**

- This stage is the combination of the Shuffle stage and the Reduce stage.
- The reducer's job is to process the data that comes from the mapper.
- After processing, it produces a new set of output, which will be stored in the HDFS.



- The Input is a set of Data, which is provided to the Map() function
- The Map() function here breaks this data blocks into **Tuples** that are a key-value pair.
- These key-value pairs are now sent as input to the Reduce().
- The Reduce() function then combines this broken Tuples or key-value pair based on its key value and form set of Tuples, and perform some operation like sorting, summation type job, etc.
- The combined result of operation is then sent to the final Output Node to get the final output.

- The data processing is always done in Reducer depending upon the business requirement of that industry.
- This is How First Map() and then Reduce is utilized one by one.

Map Task

- **RecordReader :**

- The purpose of *recordreader* is to break the records.
- *Recordreader* is responsible for providing key-value pairs in a Map() function.
- The key is actually, its locational information and value is the data associated with it.

- **Map**

- A map is a user-defined function whose work is to process the Tuples obtained from record reader.
- The Map() function either does not generate any key-value pair or generate multiple pairs of these tuples.

- **Combiner:**

- Combiner is used for grouping the data in the Map workflow.
- The intermediate key-value that are generated in the Map is combined with the help of this combiner.

- **Partitioner**

- Partitioner is responsible for partitioning key-value pairs generated in the Mapper Phases using a user-defined condition, which works like a hash function.
- The total number of partitions is same as the number of Reducer tasks for the job

Reduce Task

- **Shuffle and Sort:**
 - The process in which the Mapper generates the intermediate key-value and transfers them to the Reducer task is known as *Shuffling*.
 - Using the Shuffling process the system can sort the data using its key.
 - Once some of the Mapping tasks are done Shuffling begins and does not wait for the completion of the task performed by Mapper.

- **Reduce:**

- The main function of the Reduce is to gather the Tuple generated from Map and then perform aggregation type of process on those key-value depending on its key element.

- **OutputFormat:**

- Once all the operations are performed, the key-value pairs are written into the file with the help of record writer.
- Key-value pairs are written such that, each record in a new line, and the key and value in a space-separated manner.

How MapReduce Organizes Work?

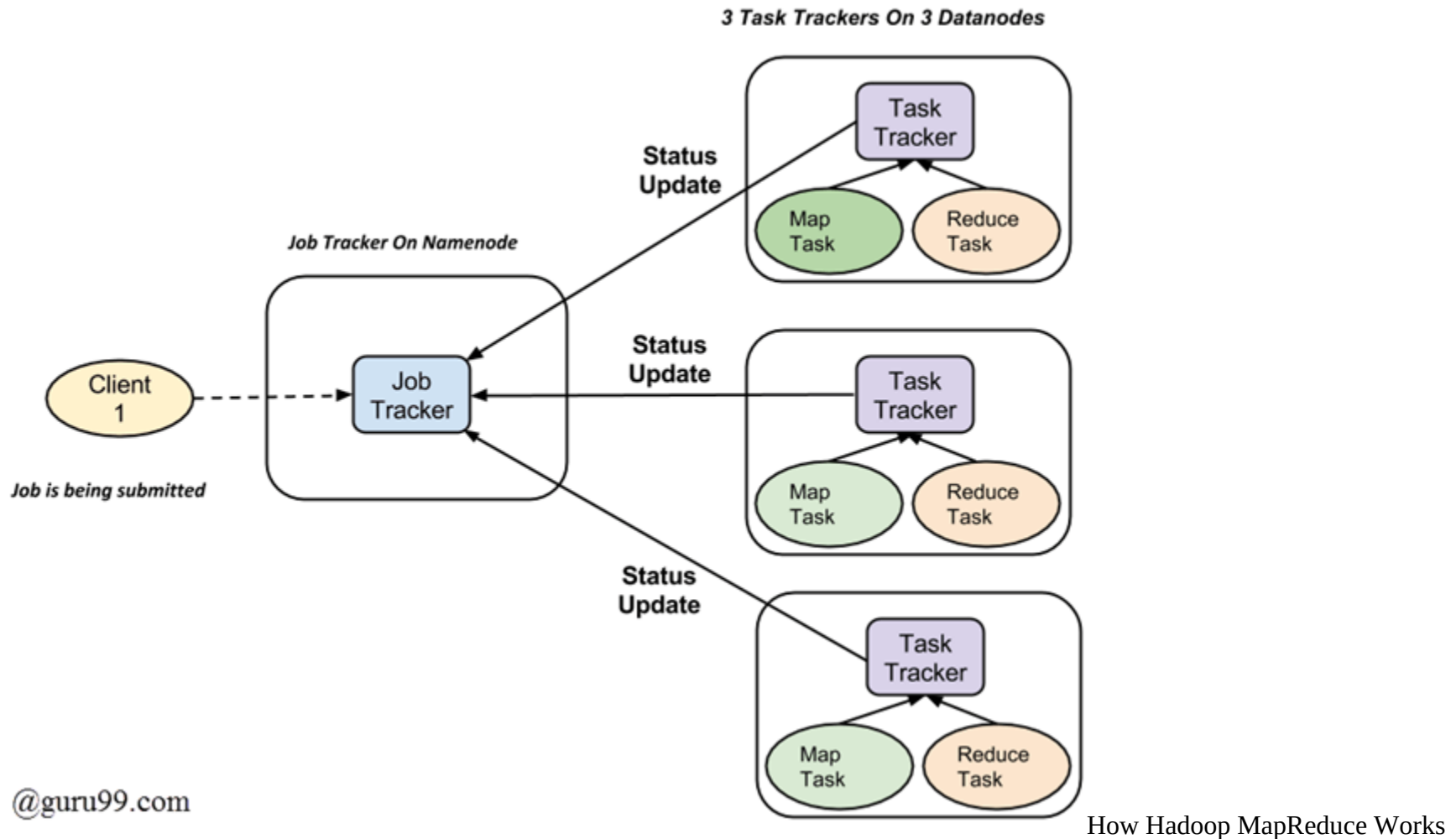
Hadoop divides the job into tasks. There are two types of tasks:

1. **Map tasks** (Splits & Mapping)
2. **Reduce tasks** (Shuffling, Reducing)

As mentioned above the complete execution process (execution of Map and Reduce tasks, both) is controlled by two types of entities called a

1. **Jobtracker**: Acts like a **master** (responsible for complete execution of submitted job)
2. **Multiple Task Trackers**: Acts like **slaves**, each of them performing the job

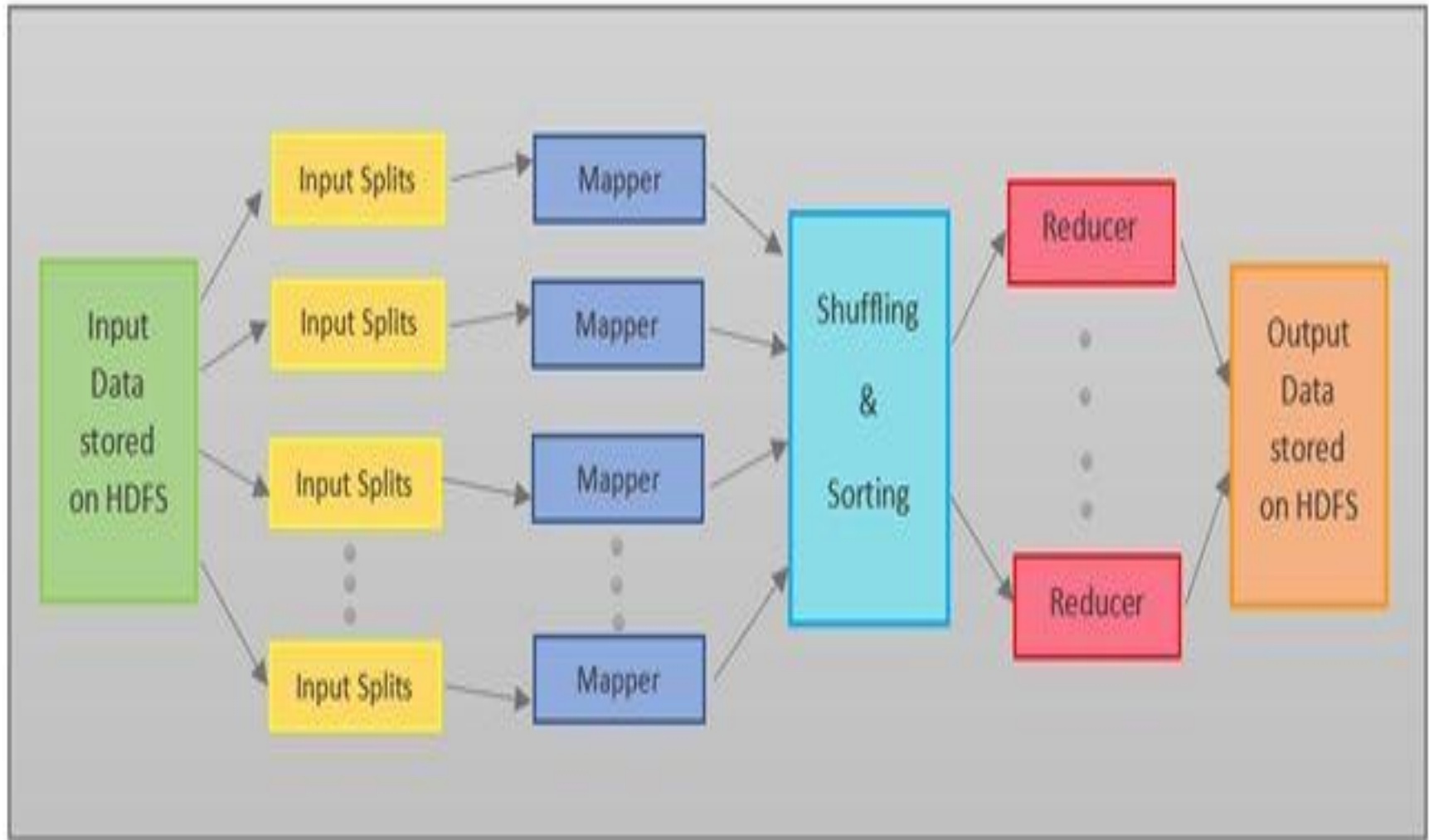
For every job submitted for execution in the system, there is one **Jobtracker** that resides on **Namenode** and there are **multiple tasktrackers** which reside on **Datanode**.



- A job is divided into multiple tasks which are then run onto multiple data nodes in a cluster.

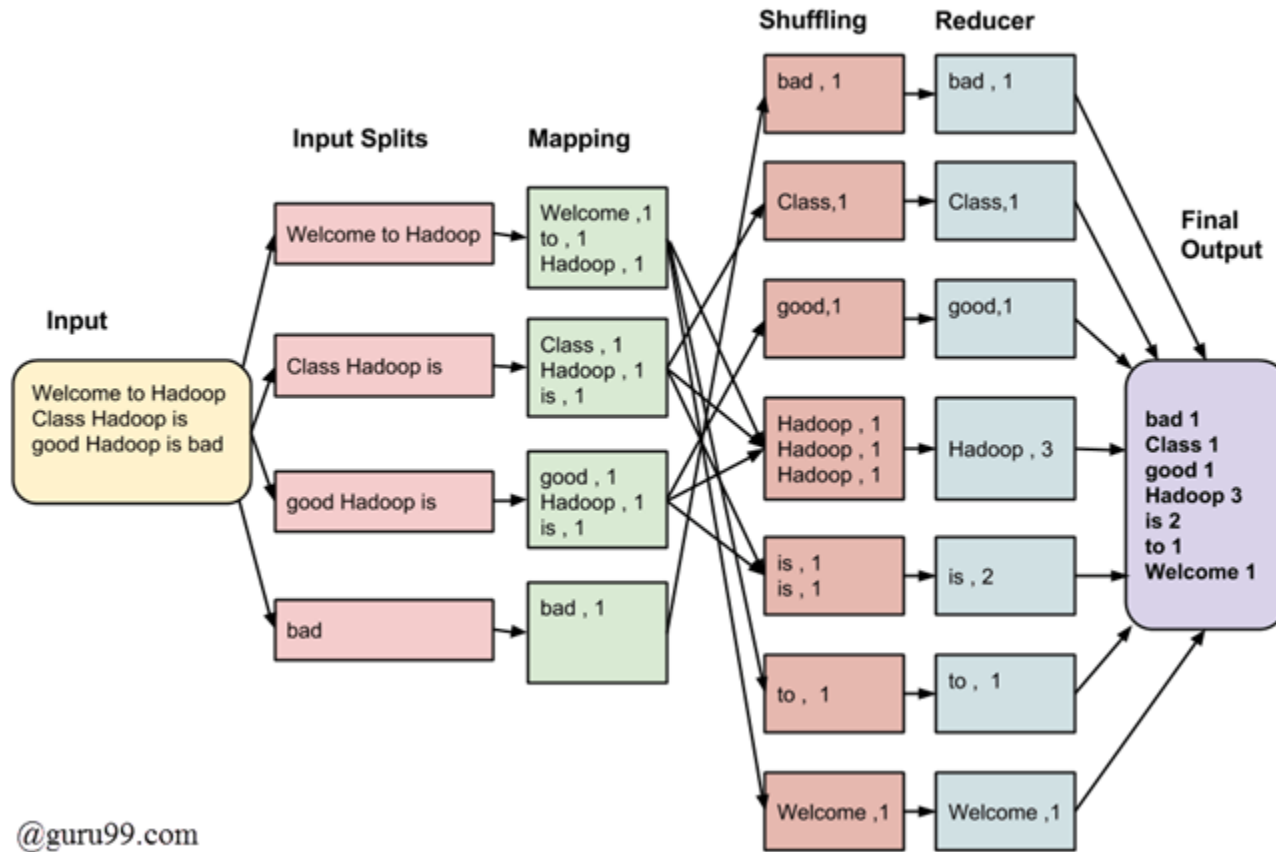
- It is the responsibility of job tracker to coordinate the activity by scheduling tasks to run on different data nodes.
- Execution of individual task is then to look after by task tracker, which resides on every data node executing part of the job.
- Task tracker's responsibility is to send the progress report to the job tracker.
- In addition, task tracker periodically sends 'heartbeat' signal to the Jobtracker so as to notify him of the current state of the system.
- Thus job tracker keeps track of the overall progress of each job. In the event of task failure, the job tracker can reschedule it on a different task tracker

MapReduce Phases



MapReduce Architecture in Big Data explained with Example

- The whole process goes through four phases of execution namely, splitting, mapping, shuffling, and reducing.
- Now in this MapReduce tutorial, let's understand with a MapReduce example–
- Consider you have following input data for your MapReduce in Big data Program
- Welcome to Hadoop Class
- Hadoop is good
- Hadoop is bad



@guru99.com

MapReduce Architecture

The final output of the MapReduce task is

Bad	●	1
-----	---	---

Class	●	1
-------	---	---

Good	●	1
------	---	---

Hadoop	●	3
--------	---	---

Is	●	2
----	---	---

To	●	1
----	---	---

Welcome	●	1
---------	---	---

The data goes through the following phases of MapReduce in Big Data

Input Splits:

An input to a MapReduce in Big Data job is divided into fixed-size pieces called input splits. Input split is a chunk of the input that is consumed by a single map.

Mapping

This is the very first phase in the execution of map-reduce program. In this phase, data in each split is passed to a mapping function to produce output values. In our example, a job of mapping phase is to count a number of occurrences of each word from input splits (more details about input-split is given below) and prepare a list in the form of <word, frequency>

Shuffling

This phase consumes the output of Mapping phase. Its task is to consolidate the relevant records from Mapping phase output. In our example, the same words are clubbed together along with their respective frequency.

Reducing

In this phase, output values from the Shuffling phase are aggregated. This phase combines values from Shuffling phase and returns a single output value. In short, this phase summarizes the complete dataset.

In our example, this phase aggregates the values from Shuffling phase i.e., calculates total occurrences of each word.

MapReduce Architecture explained in detail

- One map task is created for each split which then executes map function for each record in the split.
- It is always beneficial to have multiple splits because the time taken to process a split is small as compared to the time taken for processing of the whole input. When the splits are smaller, the processing is better to load balanced since we are processing the splits in parallel.
- However, it is also not desirable to have splits too small in size. When splits are too small, the overload of managing the splits and map task creation begins to dominate the total job execution time.
- For most jobs, it is better to make a split size equal to the size of an HDFS block (which is 64 MB, by default).

- Execution of map tasks results into writing output to a local disk on the respective node and not to HDFS.
- Reason for choosing local disk over HDFS is, to avoid replication which takes place in case of HDFS store operation.
- Map output is intermediate output which is processed by reduce tasks to produce the final output.
- Once the job is complete, the map output can be thrown away. So, storing it in HDFS with replication becomes overkill.
- In the event of node failure, before the map output is consumed by the reduce task, Hadoop reruns the map task on another node and re-creates the map output.
- Reduce task doesn't work on the concept of data locality. An output of every map task is fed to the reduce task. Map output is transferred to the machine where reduce task is

running.

- On this machine, the output is merged and then passed to the user-defined reduce function.
- Unlike the map output, reduce output is stored in HDFS (the first replica is stored on the local node and other replicas are stored on off-rack nodes). So, writing the reduce output